COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures & Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	08/18/2026	**DATE SUBMITTED:**	08/27/2026
NAME:	Baltazar, Kenji	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 6.1 **Hands-on Activity 6.1 Searching Techniques**

1. PROCEDURE OUTPUT

CODE

```
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "nodes.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main(){
    int dataset[max_size];
    srand(time(0));
    for(int i=0; i<max_size;i++){
        dataset[i] = rand();
    }
    for(int i=0; i<max_size;i++){
        std::cout<<dataset[i]<<" ";
    }
```

OUTPUT

```
21880 4304 3641 23709 24263 3258 12345 7589 28115 4967 517 29451 6276 28907 13808 16933 31
205 13050 18031 28901 8243 11947 29901 14776 8500 4988 8224 30381 18633 12182 2171 10202 5
575 18771 21093 14588 30979 12798 24202 19037 2732 30170 18865 251 10718 19813 10478 8678
14963 13277
```

OBSERVATION

When the output is run, there are random numbers created. I think these numbers can serve as the dataset for our searching activities.

Table 6-1. Data Generated and Observations.

CODE

searchingAlgo.h

```
template <typename T>
void linearSearchArr(T data[], int arrSize, T dataFind){
    int i = 0;
    while(i <= arrSize){
        if(dataFind == data[i]){
            std::cout<<"the search is a success\n";
            return;
        }
        i++;
    }
    std::cout<<"The element is not found\n";
    return;
}
```

main.cpp

```
21      int toFind;
22      std::cout<<"what value you want to search? ";
23      std::cin>>toFind;
24
25      linearSearchArr(dataset, max_size, toFind);
```

OUTPUT

```
23853 8278 12929 8958 28346 15707 24490 8987 6875 27086 9486 23287 8813 28648 15762 3380 9
113 765 9674 16490 21928 23553 22120 24520 14104 32588 763 20223 25071 3994 12502 30043 31
311 21078 14925 1075 27467 18712 4648 31876 26105 1132 23377 14297 22216 5077 10448 30830
17684 14097
what value you want to search? 8278
the search is a success
```

OBSERVATION

So, what the linear search algo do is that it checks each number in the list of those random numbers one by one until it found its target.

Table 6-2a. Linear Search for Arrays

CODE

searchingAlgo.h

```cpp
template <typename T>
void linearSearchLL(Node<T> *node, T dataFind){
    while(node != nullptr){
        if(dataFind == node->data){
            std::cout<<"the search is a success\n";
            return;
        }
        node = node->next;
    }
    std::cout<<"The element is not found\n";
    return;
}

template <typename T>
Node<T>* getMiddle(Node<T>* start, Node<T>* last){
    Node<T>* slow = start;
    Node<T>* fast = start;

    while(fast != last && fast->next != last){
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

main.cpp

```
27      Node<char>* name1  = new_node('K');
28      Node<char>* name2  = new_node('E');
29      Node<char>* name3  = new_node('N');
30      Node<char>* name4  = new_node('J');
31      Node<char>* name5  = new_node('I');
32
33      name1->next = name2;
34      name2->next = name3;
35      name3->next = name4;
36      name4->next = name5;
37
38      std::cout<<"Linear Search for Linked List result: \n";
39      linearSearchLL(name1, 'J');
40      std::cout<<"Linear Search for Linked List result: \n";
41      linearSearchLL(name1, 'x');
```

OUTPUT

```
Linear Search for Linked List result:
the search is a success
Linear Search for Linked List result:
The element is not found
```

OBSERVATION

So this linear search of ll, traversed the nodes one by one to find their target, it successfully found the existing character of J and it returns an error message when it is not found.

Table 6-2b. Linear Search for Linked List

CODE

searchingAlgo.h

```cpp
template <typename T>
T binarySearchArr(T data[], int arrSize, T findData){
    int start = 0;
    int last = arrSize - 1;
    while(start <= last){
        T middle = data[(start + last) / 2];
        if(middle == findData){
            std::cout << "the search is a success\n";
            return middle;
        } else if(findData < middle){
            last = (start + last) / 2 - 1;
        } else{
            start = (start + last) / 2 + 1;
        }
    }
    std::cout << "The element is not found\n";
    return T();
}
```

main.cpp

```cpp
int toFind2;
std::cout<<"what value you want to search? ";
std::cin>>toFind2;
binarySearchArr(dataset, max_size, toFind2);
```

OUTPUT

```
what value you want to search? J
The element is not found
```

OBSERVATION

This is the binary search algo where it tried to find the target by splitting the sets of data in half on repeat. Though it did not find the element as there is no J in the random numbers data set.

Table 6-3a. Binary Search for Arrays

CODE

searchingAlgo.h

```
template <typename T>
void binarySearchLL(Node<T>* start, T findData){
    Node<T>* last = nullptr;

    while(start != last){
        Node<T>* middle = getMiddle(start, last);
        if(middle == nullptr)
        break;

        if(findData == middle->data){
            std::cout << "the search is a success\n";
            return;
        } else if(findData < middle->data){
            last = middle;
        } else{
            start = middle->next;
        }
    }
    std::cout << "The element is not found\n";
}
```

main.cpp

```
std::cout<<"Binary Search for Linked List result: \n";
binarySearchLL(name5, 'I');
```

OUTPUT

```
Binary Search for Linked List result:
the search is a success
```

OBSERVATION

The same principle for the binary search array, so this search goes to the middle of the ll and narrows it down from there. And as seen on the output, it has successfully found its target but if the name5 is changed to like name4 then it says that it is not found.

Table 6-3b. Binary Search for Linked List

2. ANSWER TO SUPPLEMENTARY ACTIVITY

CODE

Problem 1

supplementary.h

```
template <typename T>
int linearSearchArr2(T data[], int arrSize, T dataFind){
    for(int i = 0; i < arrSize; i++){
        if(dataFind == data[i]){
            std::cout << "the search is a success with " << i + 1 << " comparisons\n";
            return i + 1;
        }
    }
    std::cout << "The element is not found\n";
    return arrSize;
}
```

```
template <typename T>
int linearSearchLL2(Node<T> *node, T dataFind){
    int ll = 0;
    while(node){
        ll++;
        if(dataFind == node->data){
            std::cout << "the search is a success with " << ll << " comparisons\n";
            return ll;
        }
        node = node->next;
    }
    std::cout << "The element is not found\n";
    return ll;
}
```

main2.cpp

```
int array1[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
int num1 = 10;
std::cout << "Linear Search for Array result: ";
linearSearchArr2(array1, num1, 18);

Node<int> *head1 = new_node(array1[0]);
Node<int> *temp1 = head1;
for(int i = 1; i < num1; i++){
    temp1->next = new_node(array1[i]);
    temp1 = temp1->next;
}
std::cout << "Linear Search for Linked List result: ";
linearSearchLL2(head1, 18);
```

Problem 2

supplementary.h

```cpp
template <typename T>
int countArr(T data[], int arrSize, T dataFind){
    int count = 0;
    for(int i = 0; i < arrSize; i++){
        if(data[i] == dataFind){
            count++;
        }
    }
    std::cout << count << " repeat\n";
    return count;
}
```

```cpp
template <typename T>
int countLL(Node<T> *node, T dataFind){
    int count = 0;
    while(node){
        if(node->data == dataFind){
            count++;
        }
        node = node->next;
    }
    std::cout << count << " repeat\n";
    return count;
}
```

main2.cpp

```cpp
std::cout << "Array Count result: ";
countArr(array1, num1, 18);
std::cout << "Linked List Count result: ";
countLL(head1, 18);
```

Problem 3

searchingAlgo.h

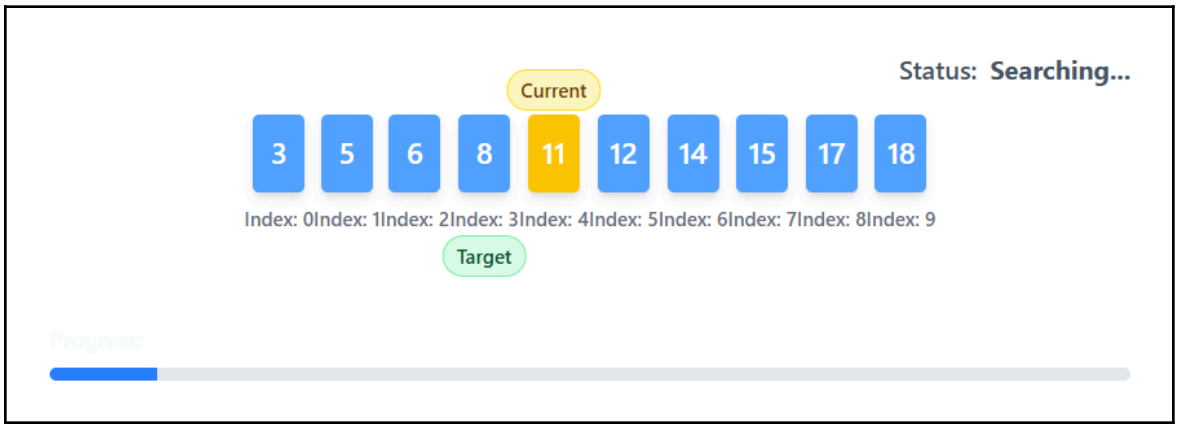
```
template <typename T>
T binarySearchArr(T data[], int arrSize, T findData){
    int start = 0;
    int last = arrSize - 1;
    while(start <= last){
        T middle = data[(start + last) / 2];
        if(middle == findData){
            std::cout << "the search is a success\n";
            return middle;
        } else if(findData < middle){
            last = (start + last) / 2 - 1;
        } else{
            start = (start + last) / 2 + 1;
        }
    }
    std::cout << "The element is not found\n";
    return T();
}
```

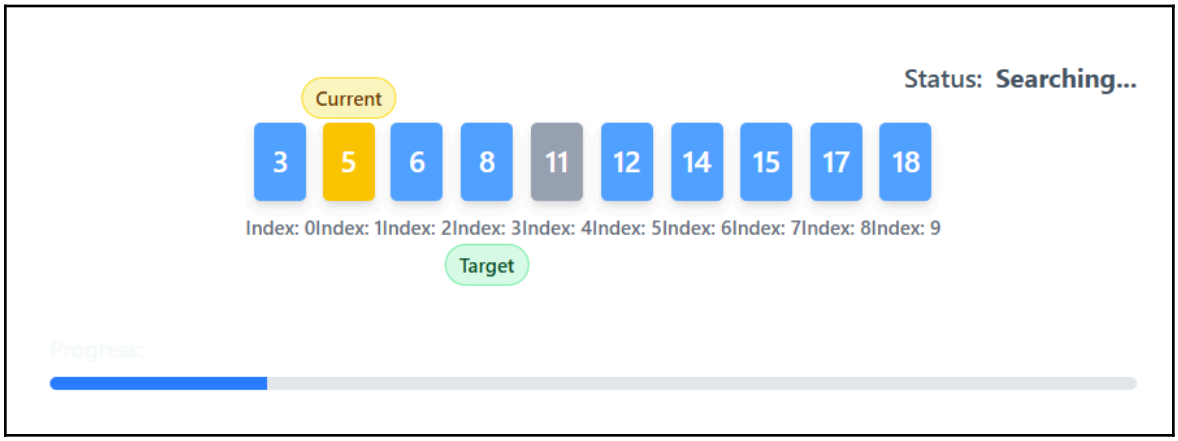
main2.cpp

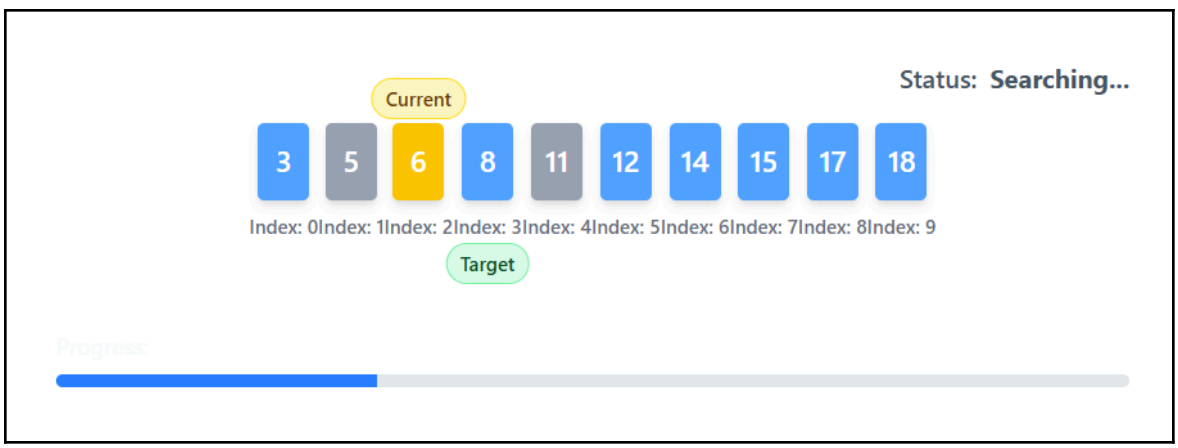
```
int array3[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
int num3 = 10;
std::cout << "Binary Search for Array result: ";
binarySearchArr(array3, num3, 8);
```

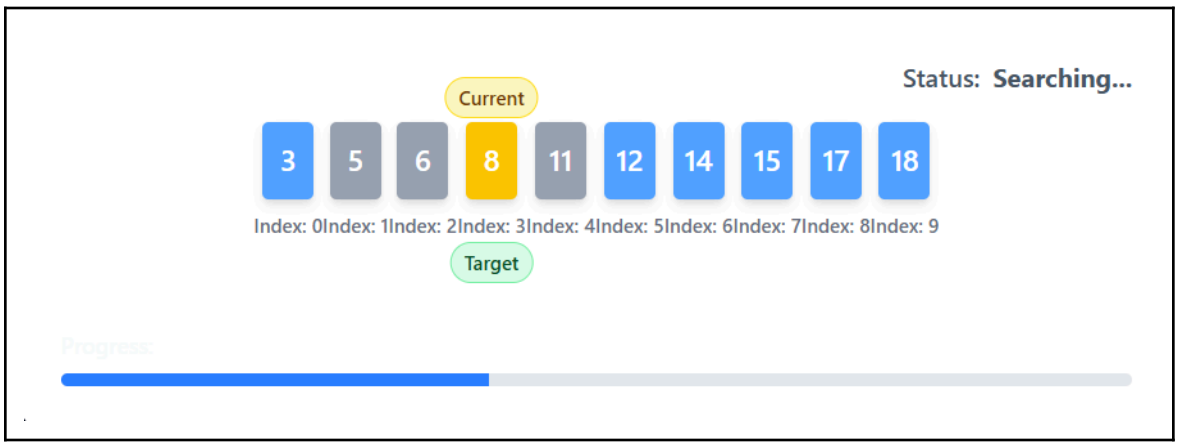
VISUALIZER


Status: Searching...
3 5 6 8 11 12 14 15 17 18
Index: 0 Index: 1 Index: 2 Index: 3 Index: 4 Index: 5 Index: 6 Index: 7 Index: 8 Index: 9
Target
Progress:


Status: Searching...
Current
3 5 6 8 11 12 14 15 17 18
Index: 0 Index: 1 Index: 2 Index: 3 Index: 4 Index: 5 Index: 6 Index: 7 Index: 8 Index: 9
Target
Progress:


Status: Searching...
Current
3 5 6 8 11 12 14 15 17 18
Index: 0 Index: 1 Index: 2 Index: 3 Index: 4 Index: 5 Index: 6 Index: 7 Index: 8 Index: 9
Target
Progress:





Source:
https://algorithm-visualizer.seancoughlin.me/searching/binarySearch?data=3%2C5%2C6%2C8%2C11%2C12%2C14%2C15%2C17%2C18&target=8&speed=3

Problem 4

supplementary.h

```cpp
template <typename T>
T BinarySearchArr2(T data[], int start, int last, T findData, int &cbnry){
    if(start > last){
        std::cout << "not found (" << cbnry << " comparisons)\n";
        return T();
    }
    int midol = (start + last) / 2;
    cbnry++;
    if(data[midol] == findData){
        std::cout << cbnry << " comparisons\n";
        return data[midol];
    }
    if(findData < data[midol]){
        return BinarySearchArr2(data, start, midol-1, findData, cbnry);
    }

    return BinarySearchArr2(data, midol+1, last, findData, cbnry);
}
```

main2.cpp

```cpp
int c2_greentea = 0;
std::cout << "Recursive Binary Search for Array result: ";
BinarySearchArr2(array3, 0, num3-1, 8, c2_greentea);
```

OUTPUT

```
Linear Search for Array result: the search is a success with 2 comparisons
Linear Search for Linked List result: the search is a success with 2 comparisons
Array Count result: 2 repeat
Linked List Count result: 2 repeat
Binary Search for Array result: the search is a success
Recursive Binary Search for Array result: 4 comparisons
```

ANALYSIS

So what I did first was make a version of linear search tracks comparison, so I used the code from the

searchingAlgo.h and modified it, changing the while to for loop so that it could loop per arrSize until it equalizes with i, then if the data is found it prints out success and returns i + 1 which is the number of comparison since comparison starts with 1 and not 0 so I added the +1. For the second task, I created the countArr & LL to determine how many repeats a value appears in the data set. Then for the third one, I just used a visualizer to visualize whats going on with the code that I used in binarySearchArr from the searchingAlgo.h. Then, I modified the binarySearchArr to become recursive to locate the elements as well as to be able to have a comparison.

3. CONCLUSION

In this activity, I learned how searching algorithms like linear and binary search work. Like, linear goes through arr or linked list one by one but the binary search is more complex like it keeps making the data set half and half until it gets the exact element. And by this hands-on, I was able to see how they perform, thanks to the visualizer as well, I was able to visualize it clearly. Overall, from procedure to supplementary, helped me understand better the logic behind both linear and binary searching and also how to apply them both in a simple coding.

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance and explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **By using claude, I was able to finish the supplementary part and also to better help me understand the concept behind each code sections. But that is just the extent of it, I was able to accomplish and observe these codes on my own.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.